sensors

MDPI

Article

Optimization of BBR Congestion Control Algorithm Based on Pacing Gain Model

Shuang Yang [1,2,3], Yuquan Tang [1], Wansu Pan [1,2,*], Huadong Wang [1], Dandan Rong [1,2] and Zhirong Zhang [1,2,3,4]

[1] Anhui Provincial Key Laboratory of Photonic Devices and Materials, Anhui Institute of Optics and Fine Mechanics, HFIPS, Chinese Academy of Sciences, Hefei 230031, China; shuangy@mail.ustc.edu.cn (S.Y.); laserway@aiofm.ac.cn (Y.T.); wanghd@aiofm.ac.cn (H.W.); rongdan@mail.ustc.edu.cn (D.R.); zhangzr@aiofm.ac.cn (Z.Z.)
[2] University of Science and Technology of China, Hefei 230026, China
[3] Guangdong Provincial Key Laboratory of Intelligent Disaster Prevention and Emergency Technologies for Urban Lifeline Engineering, Dongguan 523808, China
[4] Advanced Laser Technology Laboratory of Anhui Province, Hefei, 230037, China
* Correspondence: wansupan@mail.ustc.edu.cn; Tel.: +86-551-6559-2754

Abstract: In 2016, Google proposed a congestion control algorithm based on bottleneck bandwidth and round-trip propagation time (BBR). The BBR congestion control algorithm measures the network bottleneck bandwidth and minimum delay in real-time to calculate the bandwidth delay product (BDP) and then adjusts the transmission rate to maximize throughput and minimize latency. However, relevant research reveals that BBR still has issues such as RTT unfairness, high packet loss rate, and deep buffer performance degradation. This article focuses on its most prominent RTT fairness issue as a starting point for optimization research. Using fluid models to describe the data transmission process in BBR congestion control, a fairness optimization strategy based on pacing gain is proposed. Triangular functions, inverse proportional functions, and gamma correction functions are analyzed and selected to construct the pacing gain model, forming three different adjustment functions for adaptive adjustment of the transmission rate. Simulation and real experiments show that the three optimization algorithms significantly improve the fairness and network transmission performance of the original BBR algorithm. In particular, the optimization algorithm that employs the gamma correction function as the gain model exhibits the best stability.

Keywords: TCP congestion control; BBR; RTT fairness; pacing gain; sending rate

Citation: Yang, S.; Tang, Y.; Pan, W.; Wang, H.; Rong, D.; Zhang, Z. Optimization of BBR Congestion Control Algorithm Based on Pacing Gain Model. *Sensors* **2023**, *23*, 4431. https://doi.org/10.3390/s23094431

Academic Editors: Marco Savi, Sebastian Troia and Habib Mostafaei

Received: 9 April 2023
Revised: 27 April 2023
Accepted: 29 April 2023
Published: 30 April 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With the continuous expansion of the scale of the Internet, the number of network users and applications is rapidly increasing, and congestion has become an important issue. More congestion control studies have been undertaken to maintain the overall network's stability [1]. In 2016, Google released a congestion control algorithm (CCA) based on bottleneck bandwidth and round-trip propagation time (BBR) [2]. The BBR algorithm no longer solely uses packet loss or delay as an indicator of network congestion, but instead adjusts the sending behavior of data packets based on measured bottleneck bandwidth (BtlBw) and round-trip propagation time (RTprop). This mechanism does not reduce the congestion window (CWND) caused by a single congestion signal, avoiding a reduction in transmission rate, thereby achieving high throughput while minimizing transmission delay. This is different from traditional congestion control and has ushered in a new era of congestion control.

In various Google implementations, the BBR algorithm has been shown to significantly enhance TCP connection throughput and exhibit advantages over traditional packet loss-based algorithms in this regard [3]. Due to its excellent performance, BBR has

received a lot of attention since its release. Mathis and Mahdavi [4] claim that BBR has ushered in a new era of congestion control. However, some issues with the BBR algorithm have been discovered in some BBR evaluation reports [5–10]. For example, when BBR shares bottlenecks with Reno/CUBIC, it may lead to unfairness and overly aggressive behavior [5–7]; along with high packet loss rates in shallow buffers [8]; and bandwidth injustice across data flows with different round trip time (RTT) [9,10]. To address these issues, numerous academics have proposed optimization schemes [11,12], and Google is actively working to improve these issues. To encourage further optimization, Google has made the BBRv2 alpha version code available [13].

This article aims to provide a comprehensive analysis of the BBR congestion control mechanism, highlighting the algorithm's performance flaws, with a focus on the most significant RTT fairness issue. Several enhanced models of correlated pacing gain are proposed based on the BBR algorithm's transmission rate detection mechanism. The performance of the optimized algorithm is tested and verified simultaneously through simulation and experimental research in a natural network environment, helping to advance the BBR algorithm's development and even the investigation of next-generation CCAs. Additionally, it can guide engineers as they choose appropriate TCP congestion protocols or adjust the parameters in BBR applications.

The rest of this article is arranged as follows. Section 2 outlines the relevant research work on BBR. Section 3 introduces the algorithm of BBR and analyzes RTT fairness issues. Section 4 describes the theoretical derivation of the pacing gain model and the parameter selection process. Section 5 is the real and simulation experimental results and evaluation. Conclusions and discussions are held in Section 6.

2. Related Works

Unlike traditional packet loss-based or delay-based CCAs, BBR departs from the conventional approach of utilizing a single congestion signal to reduce transmission rates. Instead, BBR employs two control parameters: pacing rate/sending rate and CWND, enabling dynamic control of transmission rates to achieve high throughput and low latency. The Cardwell team compared the performance of the BBR with the CUBIC algorithm, demonstrating that BBR significantly improves the throughput of TCP connections. Furthermore, Google has integrated BBR into its B4 Wide Area Network (WAN) and YouTube platform, bolstering its practical application. Researchers [14–17] have conducted optimizations of the BBR algorithm, applying it to various network environments to reduce latency and improve throughput, thus enhancing network performance. As of Linux kernel version 4.19 and later, the congestion control method has been updated from CUBIC to BBR.

With the broad attention devoted to BBR algorithms, numerous studies have evaluated the performance of BBR and investigated its behavior in simulated connections or actual networks [5,18–21]. Hock et al. [5] assessed the performance of BBR in high-speed bottleneck links for the first time, including RTT fairness, latency, packet loss rate, and fairness coexisting with CUBIC. Ware et al. [18] did a performance analysis of the BBR algorithm and analyzed its advantages. Claypool et al. [19] have conducted numerous experiments on the BBR algorithm and pointed out that excessive BBR evaluation bottleneck bandwidth and RTT lead to protocol unfairness. Casas-Velasco et al. [20] analyze the performance of the CUBIC and BBR protocols in the presence of background traffic. Tao et al. [21] analyzed theoretically and found that the RTT fairness of BBRs is determined by the RTT ratio of the data flow, without involving other network parameters. Scherrer et al. [22] presented a fluid model of BBR, allowing efficient simulation under a wide variety of network settings.

Many scholars have conducted further optimization research to improve its defects. Xie et al. [23] proposed the BBR-Yinker algorithm, which can achieve high throughput and low latency over Wi-Fi and 5G networks by adjusting the pacing_gain of BBR according to the network conditions. Song et al. [24] proposed a BBR congestion window scaling

(BBR-CWS) algorithm with packet loss feedback to the size of the CWND based on packet loss feedback, balancing the throughput with other algorithms. Huang et al. [25] put forward an improved method that reduces the RTT and its jitter and improves the convergence speed of the algorithm.

To solve the problem of RTT fairness, Ma et al. [10] proposed a BBQ algorithm. This algorithm avoids delivering too many data packets into the network from long RTT flows by continuously detecting redundant queues. Additionally, it limits the maximum detection time to avoid long RTT flows from preempting short RTT flows in terms of bandwidth. Yang et al. [26] introduced an adaptive BBR algorithm, which improves fairness amongst RTT flows by varying the BBR bandwidth detection period and gain. Njogu et al. [27] proposed an algorithm that adaptively controls the CWND based on buffer queue status computation to alleviate RTT unfairness. Our previous research also proposed an improved adaptive CWND algorithm for RTT fairness in BBR [28].

Research on the BBR algorithm has continued, and Google is also continuously optimizing the BBR algorithm. In IETF-102, a preview version of BBRv2 with open-source code (called BBRv2 alpha) was proposed, encouraging researchers to dig deeper and help evaluate and improve BBRv2. This version improves the fairness of coexistence with loss-based algorithms and reduces the packet loss rate in shallow buffers. In the recent IETF-110 [29], Google proposed BBR. Swift algorithm, which uses delay as a congestion signal to achieve higher fairness and lower retransmission rate. BBRv2 can solve some fairness issues and limitations of the original BBR, but there are still some RTT fairness and intra-protocol fairness issues [22,30–33]. Kfoury et al. [31] pointed out that BBRv2's slow ability to identify available bandwidth in network contexts with erratic bandwidth leads to low link utilization. Nandagiri et al. [32] evaluated the differences between the two versions of BBR and found that there are still fairness issues in BBRv2. In addition, BBRv2 deployment in a WAN is a little complicated.

The BBR algorithm still has to be developed, and more work needs to enhance its fairness. When applying the BBR algorithm, exists a potential risk of users exploiting vulnerabilities to engage in bandwidth competition with malicious intent. Although some existing research has improved the fairness of BBR, most of the attention has been paid to inter-protocol fairness. Therefore, new approaches are still required to enhance the RTT fairness of the BBR algorithm and boost the algorithm's overall functionality.

3. BBR Congestion Control Principle

Traditional loss-based CCA increases the transmission rate by continuously increasing the transmission window (SWND) until packet loss is detected, and then quickly reduces the SWND. The final convergence occurs at the point where packet loss occurs after buffer overflow, as shown in operating point B in Figure 1. However, this approach can only guarantee maximum bandwidth and cannot guarantee delay. It frequently transmits data using the entire link capacity, which can lead to buffer expansion and increased queuing delay, particularly when the buffer is very deep.

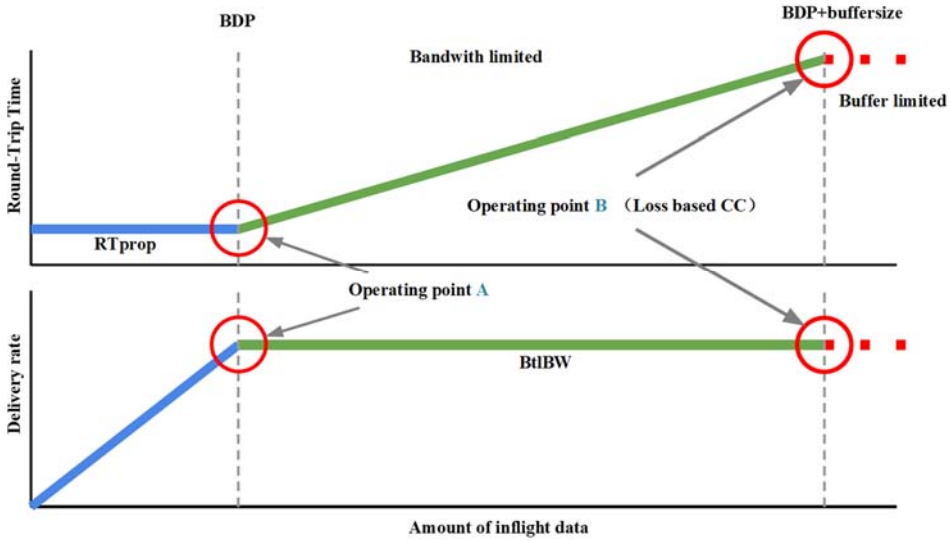


Figure 1. Changes in transfer rate and RTT with inflight: blue lines represent RTprop constraint lines, green lines represent BtlBw constraint lines, and gray dashed lines represent bottleneck buffers.

In contrast, the BBR algorithm is a novel approach that does not take cache size into account when calculating the bandwidth delay product (BDP) in the BBR model.

At the optimal operation point A in Figure 1, where the amount of data in flight is exactly 1 BDP and the SWND is equal to 1 BDP, the throughput reaches its maximum, and latency is minimized. How to accurately estimate BtlBw and RTprop becomes a key issue. BtlBw and RTprop must be continuously estimated as they can change during the course of a connection. The measurement of RTT in the TCP mechanism is as:

$$RTT_t = RTprop_t + \eta_t \tag{1}$$

where η_t represents "noise" introduced by the queue along the path; $RTprop_t$ is the physical attribute of the connection path, which only changes when the path changes. The unbiased estimate of RTprop at time t is:

$$RTprop_t = RTprop_t + \min(\eta_t) = \min(RTT_t) \quad \forall t \in [t - W_R, t] \tag{2}$$

where W_R represents the number of minutes (tens of seconds to a few minutes) during which the time window runs.

Unlike RTT, BtlBw is not measured in TCP, but it can be evaluated by measuring the delivery rate. The unbiased estimator of the maximum transfer rate in a window is BtlBw:

$$BtlBw = \max(deliveryRate_t) \quad \forall t \in [t - W_B; t] \tag{3}$$

where $deliveryRate_t$ represents the delivery rate, and the W_B is typically 6 to 10 RTTs.

BBR regards the maximum transfer rate in the last 10 RTTs as BtlBw and the minimum delay measured in the last 10 s as RTprop. Based on the measurement model, the BBR controls its transmission behavior through pacing_gain and cwnd_gain, with sending rate and CWND being:

$$Sending\ rate = pacing_gain \times BtlBw \tag{4}$$

$$CWND = cwnd_gain \times BtlBw \times RTprop \tag{5}$$

The BBR algorithm has four control states: StartUp, Drain, ProbeBW, and ProbeRTT. As shown in Figure 2.

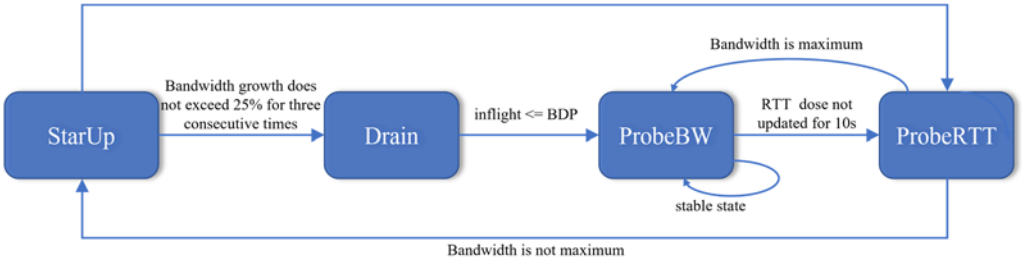


Figure 2. BBR state machine.

The StartUp state in BBR is very similar to the slow start phase. During this state, both the *pacing_gain* and *cwnd_gain* are set to 2/ln2 (approximately 2.85) to increase their transmission rate and allow the sender to detect the maximum available bandwidth. When the newly estimated BtlBw does not exceed 1.25 times the previous BtlBw estimate for three consecutive times, the algorithm assumes that the link has been filled and the state changes to Drain. In the Drain state, the *pacing_gain* = ln2/2 gradually reduces the transmission rate until the data packet in the link matches 1 BDP, at which point the state changes to ProbeBW.

Most data flows are in the ProbeBW state, with eight pacing gain cycles of different values (*pacing_gain*[] = [1.25; 0:75; 1; 1; 1; 1; 1; 1; 1]), and the duration of each cycle is one RTprop. During the detection rising phase, *pacing_gain* = 1.25 is used to increase the transmission rate and detect more available bandwidth. During the descent phase, *pacing_gain* = 0.75 is used to eliminate excessive queues accumulated. Finally, pacing_gain = 1 is used for six cycles to maintain a stable transmission rate. Meanwhile, the *cwnd_gain* is set to a constant value of 2 to ensure sufficient packets are sent during the probe phase.

If the minimum delay RTprop is not sampled again within 10 s, the BBR mechanism assumes that the link is in a congested state and switches from the ProbeBW state to the next ProbeRTT state. In this state, a data evacuation operation is performed, and the CWND is set to 4MSS. New data packets are injected into the link until the size of the valid data packet is less than 4MSS, and the new minimum delay RTprop value is sampled. After the timeout, the algorithm returns to the StartUp or ProbeBW state based on the network load.

4. Optimization Algorithm Based on Pacing Gain Model

4.1. Design Motivation

The BBR periodically detects more bandwidth at 1.25 times the BtlBw and queues up excess traffic of 0.25 BDP. Short RTT flows have a smaller estimated BDP than long RTT flows, so their proportion in queue backlog is relatively small, ultimately leading to a decrease in the actual transmission rate. The persistent low transmission rates can cause a decrease in the BtlBw estimate, which further reduces the queue share and initiates a feedback cycle. Long RTT flows have a larger estimated BDP value, so the proportion of persistent queues will increase as the short RTT flows yield, allowing them to transmit data at a higher transmission rate than short RTT flows. Due to the preemption of long RTT flows, it will be difficult for short RTT flows to detect available bandwidth, and their eventual bandwidth tends to be 0.

The throughput of short RTT flows significantly decreases once a queue backlog is formed, while the throughput of long RTT flows slightly decreases. Ma [10] measured the instantaneous queue backlog of 10 ms and 50 ms RTT flows and analyzed its impact on throughput, as depicted in Figure 3. When a competitor (50 ms RTT flow) enters the ProbeRTT state, the 10 ms RTT flow quickly consumes the buffer share, enabling it to detect additional bandwidth. As the queue is emptied by the 10 ms RTT flow and it attempts to identify new bandwidth, the 50 ms RTT flow reverts to the ProbeBW state. The preemption of 50 ms RTT flows poses a challenge to achieving higher transmission rates for short

RTT flows. To address the RTT fairness issue in BBR, it is necessary to mitigate the impact of queue backlog on the transmission rate.

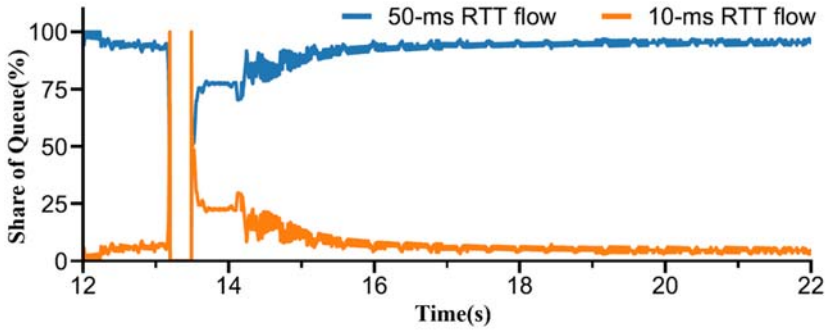


Figure 3. Queue share of two BBR flows from 12 s to 22 s [10].

In an ideal scenario, the pacing gain coefficient for the ProbeBW state can be calculated based on the absolute value of the transmission rate, replacing the fixed values of 1.25 and 0.75. Different RTT flows can adjust the increase or decrease of the gain coefficient through their respective transmission rates. The above method is not applicable in the actual process since the BBR algorithm measures and calculates the transmission rate on each end-to-end host, making the bandwidth identification between various RTT flows independent.

To ensure fair competition between different RTT flows, the pacing gain of each BBR flow is adaptively adjusted to guide flows to share bandwidth fairly. When the bandwidth ratio of the BBR flow is lower, the upward and downward gains in the ProbeBW state are larger, enabling data flows to rapidly increase throughput during the upward bandwidth detection phase and slowly decrease throughput during the downward emptying phase. Conversely, when the proportion of bandwidth occupied by BBR flows is higher, the upward and downward gains are smaller, ensuring that the flow's throughput increases slowly during the upward detection phase and decreases rapidly during the downward emptying phase, thereby providing additional bandwidth for other flows.

Based on the above analysis, an improved bandwidth detection strategy is proposed for the ProbeBW state. The original BBR's fixed pacing gain coefficients of 1.25 and 0.75 are replaced by the pacing gain coefficients P_{up} and P_{down}. If the inflight is less than 1.25 BDP, different RTT flows increase the transmission rate based on the P_{up}, actively detecting more available bandwidth. As the transmission rate increases, the amount of data in the link gradually exceeds the bottleneck transmission capacity, forming a queue at the bottleneck buffer. If the inflight in the link is greater than 1.25 BDP and packet loss occurs, the transmission rate is reduced based on the P_{down}, thereby reducing the number of data packets entering the link in the next cycle.

4.2. Construction of Pacing Gain Model

4.2.1. Model Derivation

Assume that n different flows are passing through a bottleneck link with a bandwidth of C, and d_i(t) denotes the delivered rate of the *flow*$_i$ $I \in [1,n]$ at time t. According to Equation (3), the estimated maximum bandwidth of the data flow at time t can be obtained:

$$BtlBw_i(t) = max\left(d_i(t)\right)\left(T \in \left[t - 10RTT;\ t\right]\right) \tag{6}$$

In an ideal state, $d_1 + d_2 + \cdots + d_n = C$. Let T_i(t) represent the RTT of *flow*$_i$ at time t, which is determined by:

$$T_i(t) = \frac{q_i(t)}{C} + p_i \tag{7}$$

where $q_i(t)/C$ is the queuing delay of *flow*$_i$ and p_i is the RTprop of *flow*$_i$.

Let I_i(t) stand for the inflight, which is the bottleneck link's upper limit, and be calculated as follows:

$$I_i(t) = d_i(t) \times T_i(t) = d_i(t) \times (\frac{q_i(t)}{C} + p_i) \tag{8}$$

During the first upward detection phase, flows can achieve a gain of 1.25 times, so the maximum delivered rate at time *t* is:

$$\max\left(d_i(t)\right) = \frac{\max\left(\mathrm{I}_i(t)\right)}{T_i} = \frac{1.25 \times p_i \times BtlBw_i(t-\Delta t)}{T_i} \tag{9}$$

The detection period of the BBR is 8RTprop, and the bandwidth estimation of *flow*$_i$ in the new round is updated to:

$$BtlBw_i(t) = \frac{1.25 \times p_i \times BtlBw_i(t-8p_i)}{T_i} \tag{10}$$

The queuing delay is the same for flows that are constrained by the same bottleneck queue. Once a queue is generated, the actual bandwidth cannot be increased by the expected 1.25 times, and the actual gain is $\frac{1.25 \times RTprop}{T}$ (less than 1.25). This demonstrates that the effective gain coefficient increases with increasing RTT. The RTT fairness of the BBR algorithm can be improved by adjusting the gain coefficient. So, we attempt to construct a negative feedback model using RTT to constrain the P_{up} and P_{down}. Let ω be the percentage of the current RTT and the maximum RTT:

$$\omega_i = \frac{T_i}{T_{max}} \ \left(\omega_i \in (0,1]\right) \tag{11}$$

where T_{max} is the maximum RTT of the flow over this connection, and $\omega = 1$ only when the bottleneck link capacity and buffer are fully occupied.

To optimize the BBR algorithm, it is necessary to carefully select the values of Pup and Pdown for the ProbeBW state based on ω. For P_{up}, the correlation function $P_{up}(\omega)$ should be a concave curve. When ω is large, the $P_{up}(\omega)$ should be lowered slowly. When ω is small, the $P_{up}(\omega)$ should be sensitive to link utilization and rapidly decline. This approach helps to minimize bandwidth usage for high-priority flows while maximizing available bandwidth for lower-priority flows. For P_{down}, the correlation function $P_{down}(\omega)$ should be a convex curve. When ω is large, the $P_{down}(\omega)$ should decrease rapidly, and when ω is small, the $P_{down}(\omega)$ should decrease slowly. It is important to note that the chosen functions for P_{up} and P_{down} should be of low complexity, as they will need to be implemented within the BBR algorithm.

4.2.2. Function Selection

To meet the functional requirements outlined in Section 4.2.1, three groups of functions were developed. The first group, which we refer to as BBR-A, uses the inverse proportion function and the cos function to construct the $P_{up}(\omega)$ and $P_{down}(\omega)$:

$$a = \begin{cases} P_{up}(\omega_i) = \frac{2}{\omega_i + 1} \ \left(P_{up}(\omega_i) \in [1,2]\right) \\ P_{down}(\omega_i) = \cos(\omega_i) \ \left(P_{down}(\omega_i) \in (0.5,1]\right) \end{cases} \tag{12}$$

This approach replaces the fixed pacing gains of 1.25 and 0.75 in the original BBR algorithm with the more flexible P_{up} and P_{down}. BBR-A sets P_{up} to between [1,2] to improve the flows detection, while setting P_{down} between (0.5,1] to quickly empty the queue.

However, the asymmetric $P_{up}(\omega)$ and $P_{down}(\omega)$ in the BBR-A algorithm can cause data volume imbalances of 0.5 BDP size on the link, which may lead to queue backlog. To address this issue, a symmetric function $P_{up}(\omega)$ and $P_{down}(\omega)$ is constructed. Meanwhile, to alleviate the radical detection in BBR-A, the P_{up} and P_{down} value ranges from [1,1.5] and [0.5,1], respectively. This improved algorithm is abbreviated as BBR-B:

$$b = \begin{cases} P_{up}(\omega_i) = \dfrac{3}{\omega_i + 2} \ \left(P_{up}(\omega_i) \in [1,1.5)\right) \\ P_{down}(\omega_i) = \dfrac{3}{\omega_i - 3} + 2 \ \left(P_{down}(\omega_i) \in [0.5,1)\right) \end{cases} \tag{13}$$

Furthermore, it is discovered that gamma correction is frequently employed in image processing to smooth images when looking for functions. To optimize the pacing gain mode, we combine gamma correction with a pacing gain model, adjusting the gamma correction function parameters to better meet the actual needs of the functions:

$$c = \begin{cases} P_{up}(\omega) = 1.5 - 0.5 \times \omega^{0.25} \ \left(P_{up}(\omega) \in [1,1.5]\right) \\ P_{down}(\omega) = 1 - 0.5 \times \omega^{4} \ \left(P_{down}(\omega) \in [0.5,1]\right) \end{cases} \tag{14}$$

This newly established set of P_{up} and P_{down} functions forms a new optimization algorithm BBR-C. As shown in Figure 4, the change trends of all three functions meet the set requirements. As ω increases, $P_{up}(\omega)$ slowly decreases, and the decreasing trend becomes slower. For $P_{down}(\omega)$, as ω increases, $P_{down}(\omega)$ slowly decreases, but the downward trend accelerates. The P_{up} and P_{down} built using gamma correction among them more closely adhere to the variation rule of ideal pacing gain.

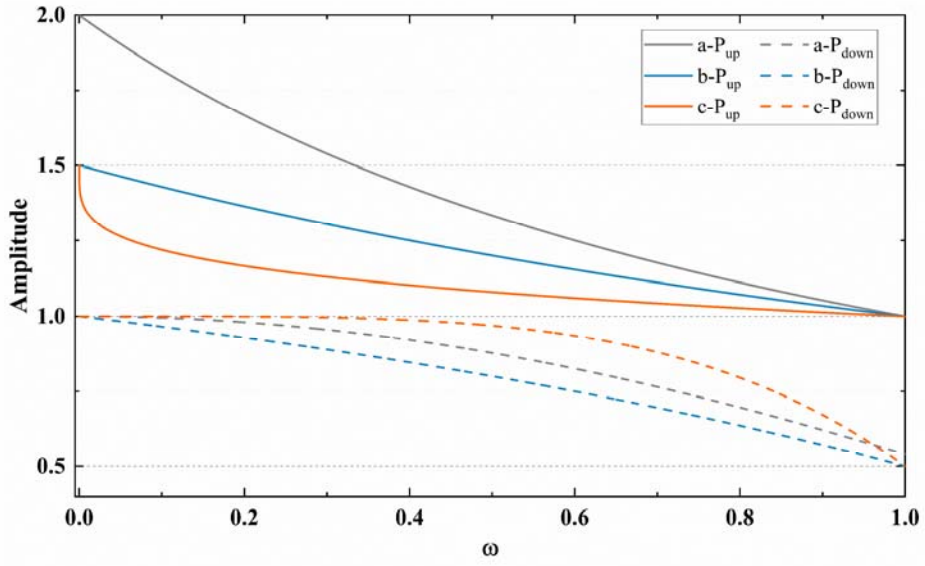


Figure 4. The variation curves of the P_{up} and P_{down} function.

5. Performance Evaluation

To evaluate the performance of the CCAs, we constructed small-scale real and a variety of simulated network scenarios using cloud servers and NS3. We compared the performance of BBR-A, BBR-B, and BBR-C algorithms with the original BBR, BBR variant algorithms BBQ, and BBRv2. In the small-scale real network environment, we deployed four Linux hosts and four cloud servers located in Hefei, Beijing, Hong Kong, and Singapore. We conducted multiple experiments to assess the effectiveness and stability of each algorithm under different scenarios. The entire test environment is divided into LAN and

WAN. Using NS3, we also simulated various network scenarios, including different RTTs, buffer sizes, and flows. Through these experiments, we were able to determine the strengths and weaknesses of each algorithm. The topology diagram is given in Figure 5.

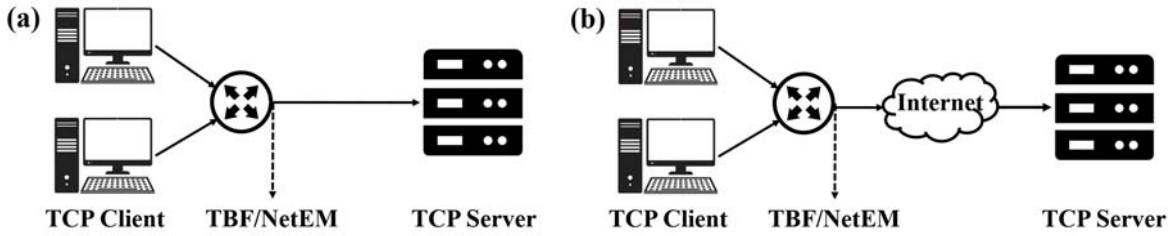


Figure 5. Real network test platform topology: (**a**) LAN testing; (**b**) WAN testing.

5.1. Network Performance under Different Buffer Sizes

First, the fairness of various optimization strategies is examined along with the throughput differences of various RTT flows in the same connection using NS3. The experimental setup used a bottleneck bandwidth of 100 Mbps and a buffer size of 5 BDP. Figure 6 displays the throughput statistics for the 10 ms and the 50 ms RTT flow for the six algorithms. The throughput of a 50 ms RTT flow for the BBR algorithm is roughly 4.7 times that of a 10 ms RTT flow. In comparison to the BBR, the throughput disparities between other algorithms were smaller, with the BBR-C algorithm having the smallest difference and being relatively stable without significant fluctuations.

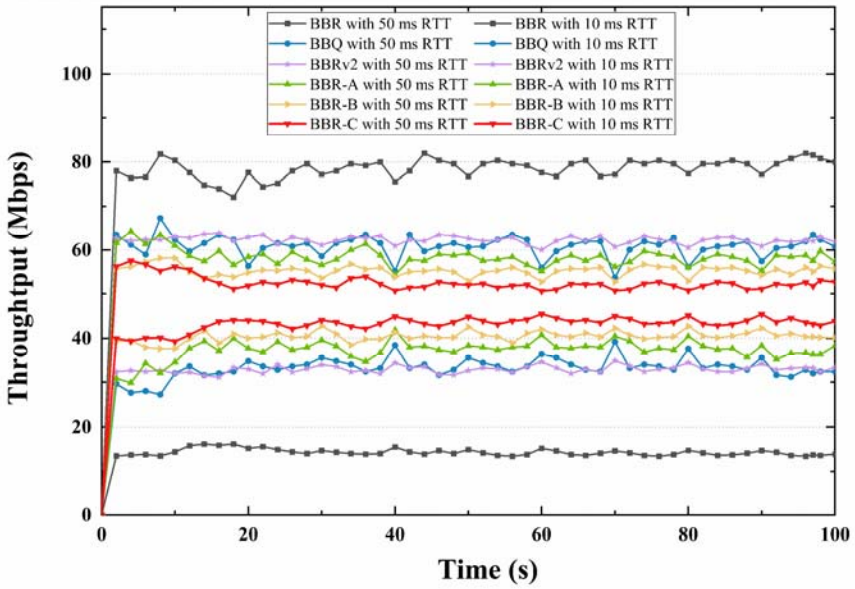


Figure 6. Throughput comparison of different congestion algorithms for 10 ms RTT flows and 50 ms RTT flows in competition.

Further, evaluate the RTT fairness of the algorithm under different buffer sizes. The Jain fairness index [34] is frequently used to measure the fairness of CCAs, and the calculation is presented in Equation (15). The fairness index, which is based on the data flow's throughput, can show how equally distributed the bandwidth is between bottlenecks among all active flows.

$$J = \left(\frac{(\sum_{i=1}^{n} x_i)^2}{n \sum_{i=1}^{n} x_i^2} \right) \quad (15)$$

where x is the throughput of flow. The Jain fairness index ranges from 0 to 1. The closer the fairness index is to 1, the better the fairness of bandwidth allocation.

We analyze the impact of buffer size on RTT fairness, setting RTT to 10 ms and 50 ms, and varying the buffer size from 0.1 to 100 BDP. Figure 7 displays the Jain fairness index of the algorithm under various buffer sizes. The 50 ms RTT flow consistently outperforms the 10 ms RTT flow, with the throughput gap widening as buffer size increases. Between 5–9 BDP buffer sizes, BBRv2's 10 msRTT and 50 msRTT flows undergo a throughput crossover, leading to an increase in the fairness index. BBR exhibits the lowest fairness index among the five algorithms while our proposed algorithms maintain a fairness index above 0.964, indicating equal bandwidth sharing between two flows. BBR-C performs best among the three optimization algorithms.

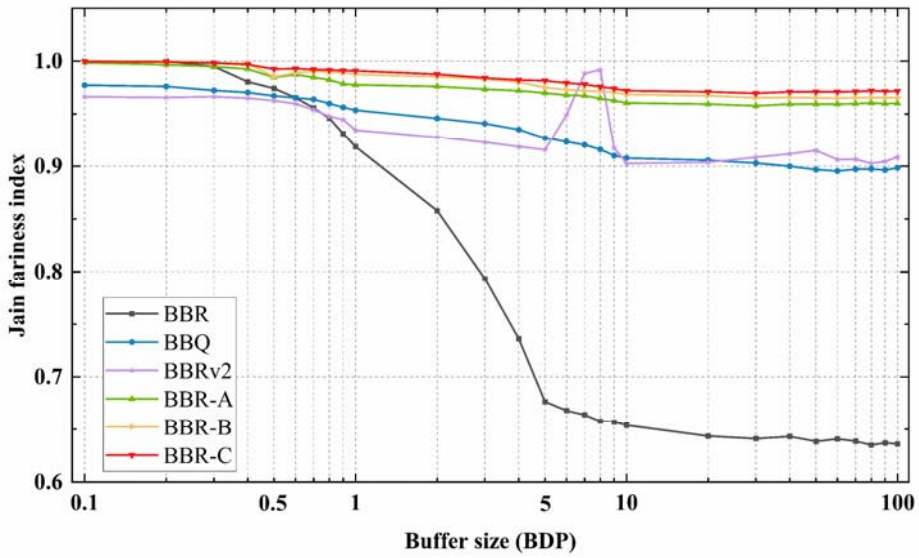


Figure 7. The Jain fairness index of congestion algorithms for 10 ms RTT flows and 50 ms RTT flows competing under different buffer sizes.

Next, evaluate the impact of the RTT ratio on algorithm RTT fairness under two buffer sizes. Figure 8 depicts the change of the Jain fairness index when a 10 ms RTT flow coexists with different RTTs flows. The RTT fairness of other variant algorithms has improved in different situations as compared to BBR. When the RTT ratio is less than 4, there is no significant difference in the fairness index of the other five BBR variants. Among them, the BBQ and BBRv2 perform well, but their fairness index decreases when the RTT ratio is greater than 4. The changes in BBR-A, BBR-B, and BBR-C algorithms are consistent, with BBR-C maintaining the highest fairness index at roughly 0.965. Overall, the fairness index of BBR-C is the highest among the six algorithms.

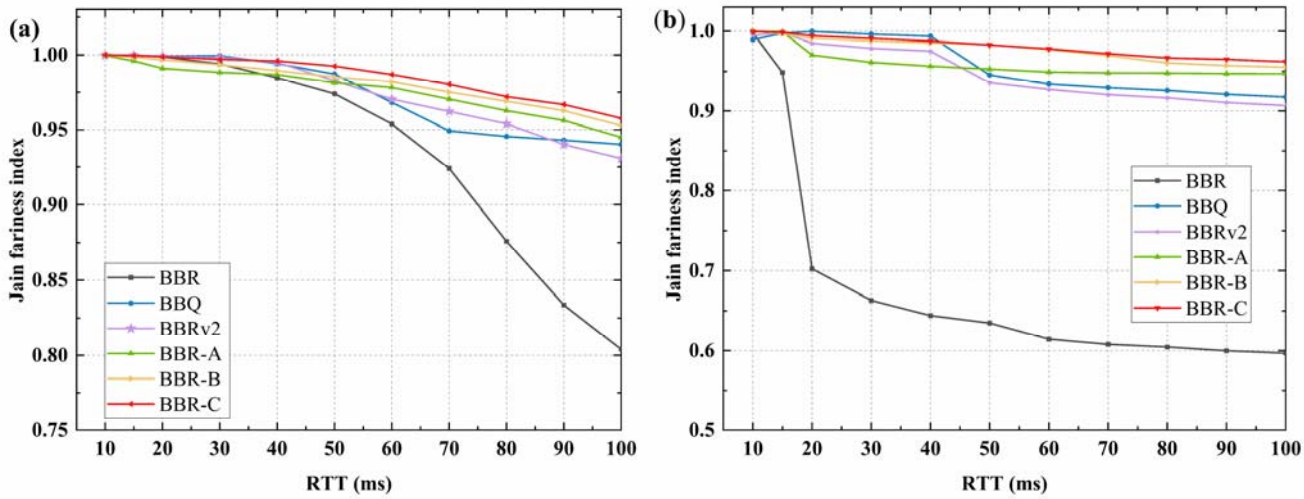


Figure 8. The Jain fairness index when 10 ms RTT flows coexist with different RTTs flows (**a**) 0.5 BDP buffer; (**b**) 5 BDP buffer.

Furthermore, the packet loss rate and delay characteristics of several CCAs were compared. A network link with a bandwidth of 100 Mbps and RTT of 30 ms was established. The buffer size ranges from 0.1 to 10 BDP. The delay and packet loss rates are shown in Figure 9. Figure 9a shows that the BBR algorithm and its variants have a longer delay than the CUBIC algorithm, which is around twice. The BBR-A algorithm has the highest latency, up to 66.7 ms. The average delay of the BBRv2 and BBR-C algorithms is close, which is between BBR (56.6 s) and BBQ (52.2 ms). As illustrated in Figure 9b, with the increase in buffer size, the packet loss rate of BBR and its variant algorithms will rapidly decrease to less than 0.1%, with BBRv2 having the lowest packet loss rate. The difference in packet loss rate among these algorithms under different buffer sizes is only 0.015%, and BBR-C is the best among the three optimization algorithms. Overall, the optimization algorithms can effectively adjust the transmission rate under different buffer sizes to achieve better network performance, and BBR-C algorithm synthesis is the best.

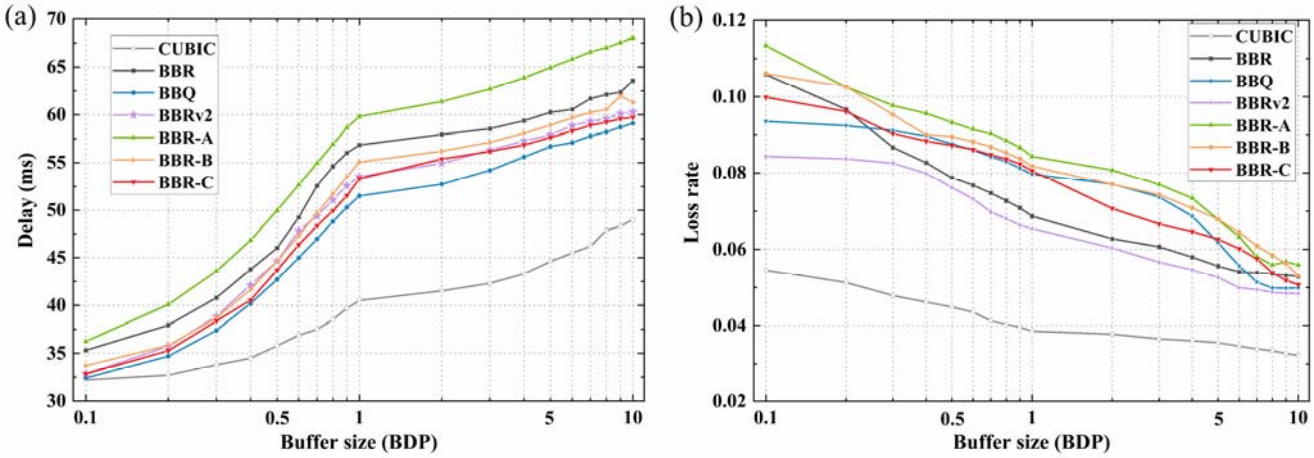


Figure 9. Performance comparison of CCAs in different buffer sizes: (**a**) delay; (**b**) packet loss rate.

5.2. Performance Testing in LAN Environments

Configure several network configurations on a local area network test platform, simulate the channel using networking simulation (NetEm) and evaluate RTT fairness. Linux TC (traffic control) to configure different network conditions for real network links, TC-tbf to set the bottleneck buffer size, and the tcpprobe to collect RTT and throughput at the

sender. For each network configuration, run the experiment at least 5 times for 200 s each time, The experiment should involve between 2 and 10 flows. Linux TC artificially increases the delay between the sender and receiver (increasing the delay by 10 ms or 50 ms). The bottleneck bandwidth is 100 Mbps, and the buffer size is set to 100 KBytes and 1 MBytes. The selected network parameter ranges such as buffer size, bottleneck bandwidth, and latency are based on commonly used values in modern networks [5,16].

Figure 10 presents the statistical results of the throughput ratio of the 10 ms and the 50 ms RTT flow under steady-state conditions. The BBR algorithm and its optimization algorithm, 50 ms RTT flows have significant advantages in different buffer sizes. Although the disadvantages of 10 ms RTT flows have been mitigated in shallow buffers (100 KBytes), their bandwidth share is still significantly lower than it should be. In comparison to the original BBR algorithm, the three improved algorithms based on BBR (BBR-A, BBR-B, and BBR-C) have improved RTT fairness to varying degrees, with the BBR-C algorithm having the highest fairness. Besides, the experimental results of the real network are fully consistent with the simulation outcomes, as observed in the previous simulation tests. Although the actual bandwidth share difference between the two RTT flows in the BBR and BBR-A algorithms is slightly larger than the simulation results, the deviation is only about 1%, which further validates the effectiveness of the simulation experimental results and the optimization algorithm.

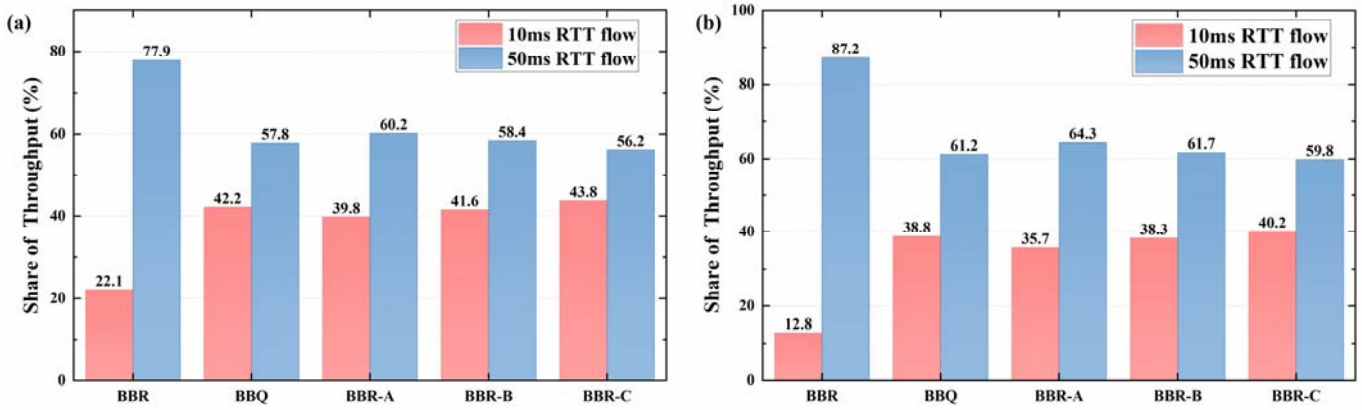


Figure 10. Throughput share of competing data flows in a LAN: (**a**) 100 KBytes; (**b**) 1 MBytes.

5.3. Performance of Long-Distance Real Network

Analyze the performance of BBR and its optimization algorithms in long-distance real network scenarios. The cloud server chose the instance path from Beijing to Singapore for network testing and the link with a maximum bandwidth of 10 Gbps.

Firstly, we measured the real throughput variations of several algorithms. The throughput versus time curve over 100 s is depicted in Figure 11. The results indicate that the throughput of BBR and its variant algorithms is significantly higher than that of CUBIC, and the throughput fluctuation of BBR-C is minimal. Subsequently, we conducted tests to assess the average latency and throughput of the algorithms to determine if they can maintain a balance between latency and throughput in long-distance real network links. Figure 12 illustrates that BBR has an average delay of 53 ms, which is twice that of the CUBIC algorithm. Meanwhile, BBR-C exhibits similar latency to BBR but significantly higher average throughput. Overall, in real network scenarios, the BBR-C algorithm has the best performance and can balance throughput and latency.

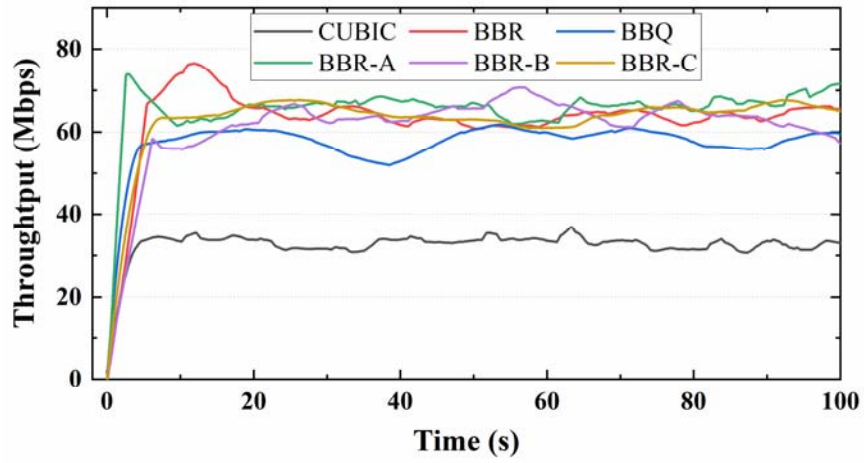


Figure 11. Throughput of congestion control protocols over time in real links.

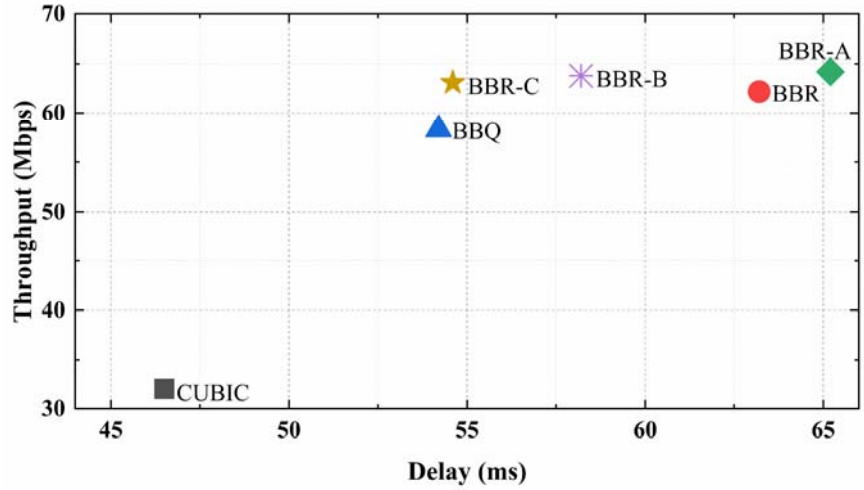


Figure 12. Comparison of throughput and delay in real links.

The RTT fairness testing will be carried out in WAN using two links from the lab to the Singapore and Hong Kong node servers, with RTTs of 10 ms and 50 ms, respectively. To facilitate comparison and analysis of the experimental results with local area networks, two buffer sizes will also be configured for the WAN test environment. The statistical results of the throughput share are shown in Figure 13.

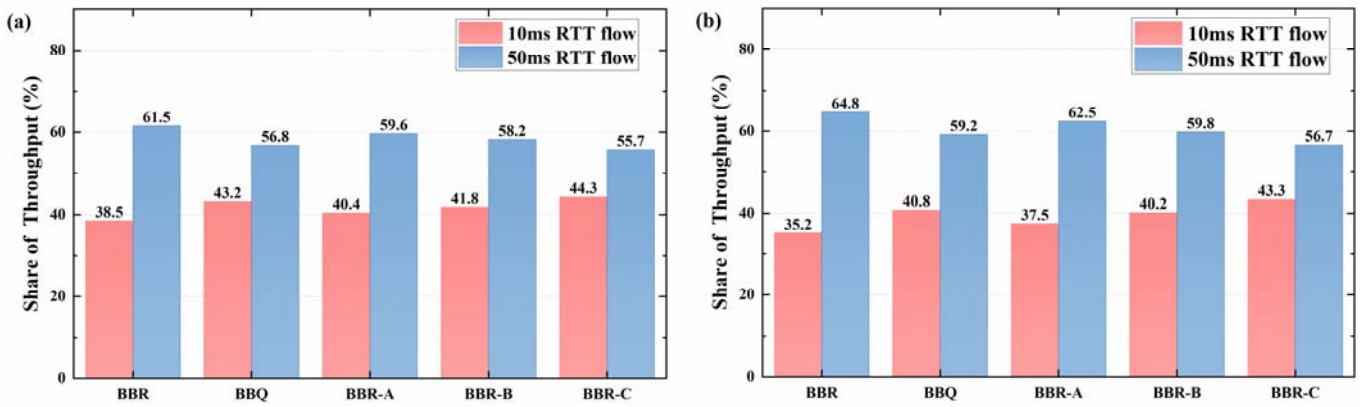


Figure 13. Throughput share of competing data flows in a wide area network: (**a**) 100 KBytes; (**b**) 1 MBytes.

The throughput share between 50 ms and 10 ms RTT flows in a wide area network (i.e., RTT unfairness) also exhibits a certain level of deviation, similar to the results of simulation experiments and local area network experiments. This further demonstrates the impact of the RTT ratio on the RTT fairness issue in BBR. However, unlike the obvious benefits of 50 ms RTT flows in LAN, the bandwidth share of 50 ms RTT flows in deep buffers has not significantly increased in WAN. Moreover, the difference in bandwidth usage between the two flows in WAN is also smaller than in simulation experiments and LAN experiments. There are two possible reasons for this result: firstly, there may be a shallow buffer in WAN between the switch and the receiver, which limits the deviation in bandwidth share caused by RTT; secondly, it may be attributed to increased uncertainty in WAN, such as layer by layer routing and processing of data packets by security devices, which makes testing unstable. This may be the main reason why the performance improvement of algorithms measured in real network experiments is slightly lower than the simulation results.

6. Conclusions

The primary goal of CCAs is to maintain network fairness. Therefore, starting from the fairness problems inherent in the BBR algorithm, this paper analyzes its detection mechanism and RTT fairness problems, establishes a theoretical model for the existing issues, and proposes pertinent optimization schemes to further improve its congestion control performance.

The impact of fixed pacing gain on RTT fairness is analyzed based on the transmission rate characteristics of the ProbeBW state in the BBR algorithm. To balance the transmission rate between different flows, a pacing gain model was established to adjust the transmission rate nonlinearly. Three gain coefficient models, including the cos function, inverse scale function, and gamma correction curve, are constructed to optimize the BBR algorithm. Based on the BBR algorithm, three optimization algorithms (BBR-A, BBR-B, and BBR-C) are proposed, which successfully enhance the fairness of the BBR algorithm, as demonstrated by performance tests in both simulation and actual network scenarios. Among these algorithms, the BBR-C algorithm provides the best congestion control impact and stabilizes the congestion control process while improving RTT fairness.

This article mainly aims to improve the RTT fairness of BBR. Future work will continue to optimize the model and the selection of key parameters, which could be adaptively adjusted using reinforcement learning and other methods. Additionally, we will keep paying attention to the latest version of BBR and conduct practical application research based on BBR and improved optimization algorithms.

Author Contributions: S.Y. proposed the idea, conducted the experiments, and wrote the paper. Y.T. provided guidance and key suggestions. W.P. supervised the entire research. H.W. guided data curation. D.R. participated in the data processing. Z.Z. contributed by guiding the paper. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the CAS & Bengbu Technology Transfer Project under Grant ZKBB202102, the Hefei Municipal Natural Science Foundation under Grant 2021021, the "Spark" Fund Project of Hefei Institutes of Physics Science, Chinese Academy Sciences under Grant YZJJ2022QN02, and the Open Project Program of Guangdong Provincial Key Laboratory of Intelligent Disaster Prevention and Emergency Technologies for Urban Lifeline Engineering under Grant 2022ZA03.

Institutional Review Board Statement: This study does not involve human or animal studies.

Informed Consent Statement: This study does not involve human or animal studies.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Nagle, J. Congestion control in IP/TCP internetworks. *ACM SIGCOMM Comput. Commun. Rev.* **1984**, *14*, 11–17.
2. Cardwell, N.; Cheng, Y.; Gunn, C.S.; Yeganeh, S.H.; Jacobson, V. BBR: Congestion-based congestion control. *ACM Queue* **2016**, *14*, 20-53.
3. Cardwell, N.; Cheng, Y.; Gunn, C.S.; Yeganeh, S.H.; Hassas, S.; Jacobson, V. BBR: Congestion-based congestion control. *Commun. ACM* **2017**, *60*, 58–66.
4. Mathis, M.; Mahdavi, J. Deprecating the TCP macroscopic model. *ACM SIGCOMM Comput. Commun. Rev.* **2019**, *49*, 63–68.
5. Hock, M.; Bless, R.; Zitterbart, M. Experimental evaluation of BBR congestion control. In Proceedings of the International Conference on Network Protocols (ICNP), Toronto, ON, Canada, 10–13 October 2017; pp. 1–10.
6. Cao, Y.; Jain, A.; Sharma, K.; Balasubramanian, A.; Gandhi, A. When to use and when not to use BBR: An empirical analysis and evaluation study. In Proceedings of the Internet Measurement Conference, Amsterdam, The Netherlands, 21–23 October 2019; pp. 130–136.
7. Sasaki, K.; Hanai, M.; Miyazawa, K.; Kobayashi, A.; Oda, N.; Yamaguchi, S. TCP fairness among modern TCP congestion control algorithms including TCP BBR. In Proceedings of the 2018 IEEE 7th International Conference on Cloud Networking (CloudNet), Tokyo, Japan, 22–24 October 2018; pp. 1–4.
8. Scholz, D.; Jaeger, B.; Schwaighofer, L.; Raumer, L.; Geyer, F.; Carle, G. Toward a Deeper Understanding of TCP BBR Congestion Control. In Proceedings of the IFIP Networking, Zurich, Switzerland, 14–16 May 2018; pp. 1–9.
9. Miyazawa, K.; Sasaki, K.; Oda, N.; Yamaguchi, S. Cycle and Divergence of Performance on TCP BBR. In Proceedings of the IEEE International Conference on Cloud Networking (CloudNet), Tokyo, Japan, 22–24 October 2018; pp. 1–6.
10. Ma, S.; Jiang, J.; Wang, W.; Li, B. Fairness of congestion-based congestion control: Experimental evaluation and analysis. *arXiv* **2017**. arXiv:1706.09115.
11. Chiariotti, F.; Zanella, A.; Kucera, S.; Claussen, H. BBR-S: A low-latency BBR modification for fast-varying connections. *IEEE Access* **2021**, *9*, 76364–76378.
12. Mishra, A.; Tiu, W.H.; Leong, B. Are we heading towards a BBR-dominant Internet? In Proceedings of the 22nd ACM Internet Measurement Conference, New York, NY, USA, 25–27 October 2022; pp. 538–550.
13. Cardwell, N.; Cheng, Y.; Yeganeh, S.H.; Swett, I.; Vasiliev, V.; Jha, P.; Seung, Y.; Mathis, M.; Jacobson, V. BBRv2: A Model-Based Congestion Control. In Proceedings of the ICCRG IETF 104th Meeting, March 2019. Available online: https://datatracker.ietf.org/meeting/104/materials/slides-104-iccrg-an-update-on-bbr-00 (accessed on 2 April 2020).
14. Verma, S.S.; Patel, R.B.; Kumar, A.; Vishwakarma, S. Analyzing QoS with Multipath Routing Using Real Flows Dynamic Queue (RFDQ) in Manet. *J. Adv. Res. Dyn. Control Syst*. **2020**, *12*, 5.
15. Prasad, A.; Verma, S.S.; Dahiya, P.; Kumar, A. A case study on the monitor mode passive capturing of WLAN packets in an on-the-move setup. *IEEE Access* **2021**, *9*, 152408–152420.
16. Haile, H.; Grinnemo, K.J.; Hurtig, P.; Brunstrom, A. RBBR: A receiver-driven BBR in quic for low-latency in cellular networks. *IEEE Access* **2022**, *10*, 18707–18719.
17. Mehbodniya, A.; Webber, J.L.; Karupusamy, S. Improving the geo-drone-based route for effective communication and connection stability improvement in the emergency area ad-hoc network. *Sustain. Energy Technol. Assess.* **2022**, *53*, 102558.
18. Ware, R.; Mukerjee, M.K.; Seshan, S.; Sherry, J. Modeling BBR's Interactions with Loss-Based Congestion Control. In Proceedings of the Internet Measurement Conference, Amsterdam, The Netherlands, 21–23 October 2019; pp. 137–143.
19. Claypool, S.; Claypool, M.; Chung, J.; Li, F. Sharing but not caring-Performance of TCP BBR and TCP CUBIC at the network bottleneck. In Proceedings of the Fifteenth Advanced International Conference on Telecommunications, Nice, France, 28 July–1 August 2019.
20. Casas-Velasco, D.M.; Granelli, F.; da Fonseca, N.L.S. Impact of Background Traffic on the BBR and CUBIC Variants of the TCP Protocol. *IEEE Netw. Lett.* **2022**, *4*, 147–151.
21. Tao, Y.; Jiang, J.; Ma, S.; Wang, L.; Wang, W.; Li, B. Unraveling the RTT-fairness Problem for BBR: A queueing model. In Proceedings of the 2018 IEEE Global Communications Conference (GLOBECOM), Abu Dhabi, United Arab Emirates, 9–13 December 2018; pp. 1–6.
22. Scherrer, S.; Legner, M.; Perrig, A.; Schmid, S. Model-based insights on the performance, fairness, and stability of BBR. In Proceedings of the 22nd ACM Internet Measurement Conference, Nice, France, 25–27 October 2022; pp. 519–537.
23. Xie, Y.; Jiang, X.; Gong, G.; Jiang, Z.; Jin, G.; Chen, H. Yinker: A flexible BBR to achieve the high-throughput and low-latency data transmission over Wi-Fi and 5G networks. *Comput. Netw.* **2023**, *222*, 109530.
24. Song, Y.J.; Kim, G.H.; Cho, Y.Z. Improvement of cyclic performance variation between TCP BBR and CUBIC. In Proceedings of the 25th Asia-Pacific Conference on Communications (APCC), Ho Chi Minh City, Vietnam, 6–8 November 2019; pp. 1–6.
25. Huang, H.; Zhu, X.; Wang, Z. Improving Latency and Bandwidth Probe of BBR Congestion Control Algorithm. *Comput. Mod.* **2022**, *10*, 113.
26. Yang, M.; Yang, P.; Wen, C.; Liu, Q.; Luo, J.; Yu, L. Adaptive-BBR: Fine-grained congestion control with improved fairness and low latency. In Proceedings of the 2019 IEEE Wireless Communications and Networking Conference (WCNC), Marrakech, Morocco, 15–18 April 2019; pp. 1–6.
27. Njogu, C.K.; Yang, W.; Njogu, H.W.; Bosire, A. BBR-With Enhanced Fairness (BBR-EFRA): A new enhanced RTT fairness for BBR congestion control algorithm. *Comput. Commun.* **2023**, *200*, 95–103.

28. Pan, W.S.; Tan, H.B.; Li. X.F.; Li. X.R. Improved RTT Fairness of BBR Congestion Control Algorithm based on Adaptive Congestion Window. *Electronics* **2021**, *10*, 615.
29. Cardwell, N.; Cheng, Y.; Yang, K.; Yeganeh, S.H.; Jha, P.; Seung, Y.; Mathis, M.; Jacobson, V. BBR updates: Internal Deployment, Code, Draft Plans. In Proceedings of the IETF 110th Meeting. Available online: https://datatracker.ietf.org/meeting/110/materials/slides-110-iccrg-bbr-updates-00.pdf (accessed on 2 April 2022).
30. Pan, W.S.; Tan, H.B.; Li, X.F.; Xu, J.L.; Li, X.R. Improvement of bbrv2 congestion control algorithm based on flow-aware ecn. *Secur. Commun. Netw.* **2022**, *2022*, 1218245.
31. Kfoury, E.F.; Gomez, J.; Crichigno, J.; Bou-Harb, E. An emulation-based evaluation of TCP BBRv2 alpha for wired broadband. *Comput. Commun*. **2020**, *161*, 212–224.
32. Nandagiri, A.; Tahiliani, M.P.; Misra. V.; Ramakrishnan, K.K. BBRvl vs BBRv2: Examining performance differences through experimental evaluation. In Proceedings of the 2020 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN), Orlando, FL, USA, 13–15 July 2020; pp. 1–6.
33. Yang, F.; Wu, Q.; Li, Z.; Liu, Y.; Pau, G.; Xie, G. BBRv2+: Towards balancing aggressiveness and fairness with delay-based bandwidth probing. *Comput. Netw.* **2022**, *206*, 108789.
34. Jain, R.K.; Chiu, D.M.W.; Hawe, W.R. *A Quantitative Measure of Fairness and Discrimination*; Eastern Research Laboratory, Digital Equipment Corporation: Hudson, MA, USA, 1984.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.